UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA

FACULTAD DE SISTEMAS Y TELECOMUNICACIONES

CARRERA:

SOFTWARE

ESTUDIANTE:

TOMALÁ GONZÁLEZ JUSTIN ARIEL

ASIGNATURA:

INTELIGENCIA DE NEGOCIOS

TEMA:

MI PRIMER DASHBOARD: ENTREGABLE 3

CURSO: 6/1

DOCENTE: ING. ANTHONY PACHAY

PERIODO

2026-2

1. Dataset y comprensión del problema

1.1 Información del dataset

Campo	Detalle
Nombre	Brazilian E-Commerce Public Dataset by Olist
Enlace de origen	https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
Descripción general	Datos reales y anonimizados de pedidos realizados en Olist, un marketplace brasileño que conecta pequeños vendedores con clientes. Incluye información del pedido, productos, pagos, entrega, vendedores y reseñas.
Cantidad de registros	99.441 pedidos y 112.650 ítems vendidos (más de 1,4 millones de filas sumando las 9 tablas).
Cantidad de columnas	52 columnas en total, distribuidas en 9 tablas.
Período temporal	Del 4 de septiembre de 2016 al 17 de octubre de 2018.

1.2 Justificación de la selección

Se seleccionó este dataset porque contiene la información necesaria para desarrollar un proyecto de Business Intelligence completo:

• Estructura relacional: nueve tablas relacionadas por claves, lo que permite construir un modelo dimensional real.

• Dimensión temporal: más de dos años de pedidos con varias marcas de tiempo por pedido.

• Medidas numéricas: precio, flete, valor de pago, cuotas y puntaje de reseña.

• Variables de análisis: estado del pedido, tipo de pago, categoría de producto y ubicación geográfica.

• Calidad de datos retadora: presenta nulos, duplicados y categorías inconsistentes que deben tratarse.

1.3 Qué representa una fila

• orders: una fila es un pedido realizado por un cliente (99.441 pedidos).

• order_items: una fila es un producto (ítem) dentro de un pedido (112.650 filas).

• order_payments: una fila es un pago asociado a un pedido.

• order_reviews: una fila es la reseña que un cliente dejó sobre un pedido.

• customers: una fila es un registro de cliente asociado a un pedido.

1.4 Problema de análisis y usuarios

Problema: Olist necesita entender el desempeño comercial, la eficiencia logística y la satisfacción de sus clientes para tomar decisiones de crecimiento, operación y gestión de vendedores.

Usuarios potenciales: Gerencia/dirección, Logística y operaciones, Experiencia del cliente, Gestión comercial de vendedores, Marketing.

2. Preguntas de negocio e indicadores

P1. ¿Cómo evolucionan las ventas en el tiempo y qué categorías de producto generan más ingresos?

P2. ¿Qué tan eficiente es la logística? ¿Se entregan los pedidos a tiempo y qué estados tienen más retrasos?

P3. ¿Qué tan satisfechos están los clientes y qué factores (retrasos) se asocian a malas reseñas?

P4. ¿Qué vendedores concentran las ventas y qué tan dependiente es el negocio de pocos vendedores?

2.1 KPI definidos

KPI	Fórmula	Pregunta
Ingresos totales (GMV) y ticket promedio	Suma de price; ticket = ingresos / pedidos	P1
Tasa de entrega a tiempo	Pedidos con entrega real ≤ fecha estimada / pedidos entregados	P2
Tiempo promedio de entrega (días)	Promedio de (entrega real − fecha de compra)	P2
Puntaje promedio de satisfacción y % reseñas 1-2	Promedio de review_score; % de scores 1 y 2	P3
Concentración de ventas por vendedor	% de ingresos que aporta el 10% superior de vendedores	P4

Criterios: los ingresos excluyen pedidos canceled y unavailable; logística y satisfacción solo sobre pedidos delivered.

3. Limpieza y transformación de datos

Las transformaciones se automatizaron en el script etl_limpieza.py. Principales acciones:

Transformación	Detalle	Resultado
Conversión de fechas	5 columnas de fecha de orders de texto a datetime	Cálculo de días de entrega y retraso
Exclusión de geolocalización	Tabla con 1M filas y 26% duplicadas no se usa	Modelo más simple
Consolidación de clientes	Agrupación por customer_unique_id	99.441 → 96.096 clientes reales
Normalización de texto	Ciudades sin tildes, estados en mayúscula	Categorías consistentes
Productos	610 sin categoría → 'sin_categoria'; 2 traducciones faltantes	Dimensión completa
Pagos	'not_defined' → 'otro'; cuotas en rangos; pago principal por pedido	dim_pago limpia
Reseñas	Una sola reseña por pedido (la más reciente)	98.673 reseñas
Medidas de entrega	dias_entrega, dias_retraso, entrega_a_tiempo	Listas para KPI
Claves sustitutas	Cada dimensión tiene clave entera + fila 0 'sin dato'	Integridad referencial

4. Modelo dimensional

Se implementó una constelación de hechos porque precio/flete están a nivel de ítem y entrega/reseña/pago a nivel de pedido.

4.1 Granularidad

fact_ventas_items: Una fila representa un ítem (producto) vendido dentro de un pedido, por un vendedor, en la fecha de compra del pedido.

fact_pedidos: Una fila representa un pedido realizado por un cliente.

4.2 Tablas de hechos

Tabla	Claves	Medidas
fact_ventas_items (112.650)	PK: (order_id, order_item_id); FK a fecha, cliente, producto, vendedor, estado	price, freight_value, total_item, cantidad
fact_pedidos (99.441)	PK: order_id; FK a fecha (compra/entrega/estimada), cliente, estado, pago	dias_entrega, dias_retraso, entrega_a_tiempo, review_score, payment_value_total, num_items

4.3 Dimensiones

Dimensión	Atributos principales
dim_fecha (822)	fecha_key, fecha, día, mes, trimestre, año, día de la semana
dim_cliente (96.097)	cliente_key, customer_unique_id, ciudad, estado, prefijo_postal
dim_producto (32.952)	producto_key, product_id, categoria_pt, categoria_en, peso, dimensiones
dim_vendedor (3.096)	vendedor_key, seller_id, ciudad, estado
dim_estado_pedido (9)	estado_key, order_status
dim_pago (10)	pago_key, tipo_pago, rango_cuotas

Decisiones de diseño: fila 'sin dato' (clave 0) en cada dimensión; dim_fecha como dimensión de rol múltiple; cancelados se cargan pero se filtran en las vistas/funciones.

4.4 Diagrama estrella (constelación de hechos)
El siguiente diagrama muestra las dos tablas de hechos (rojo) y las seis dimensiones (azul) del esquema dw, con sus claves primarias y foráneas.

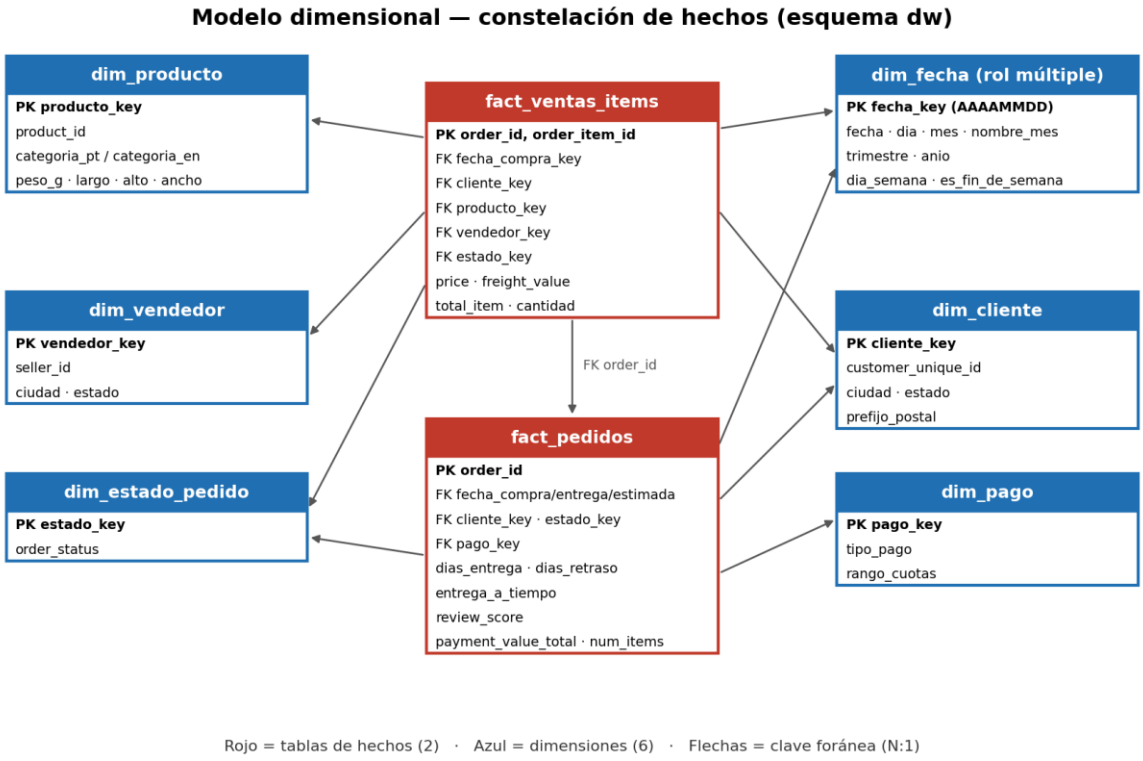


Figura 1. Diagrama del modelo dimensional implementado en PostgreSQL (esquema dw).

5. Implementación en PostgreSQL
PostgreSQL 16/17 (Docker o local). Esquema: dw. Usuario: bi_user. Base: bi_database.

Scripts:

• 01_crear_esquema.sql — crea dimensiones, hechos, PK, FK, CHECK e índices

• 02_cargar_datos.sql — COPY de los 8 CSV limpios

• 03_validaciones.sql — conteos, integridad, nulos, totales

• 04_vistas_funciones.sql — 4 vistas + 10 funciones con filtros opcionales

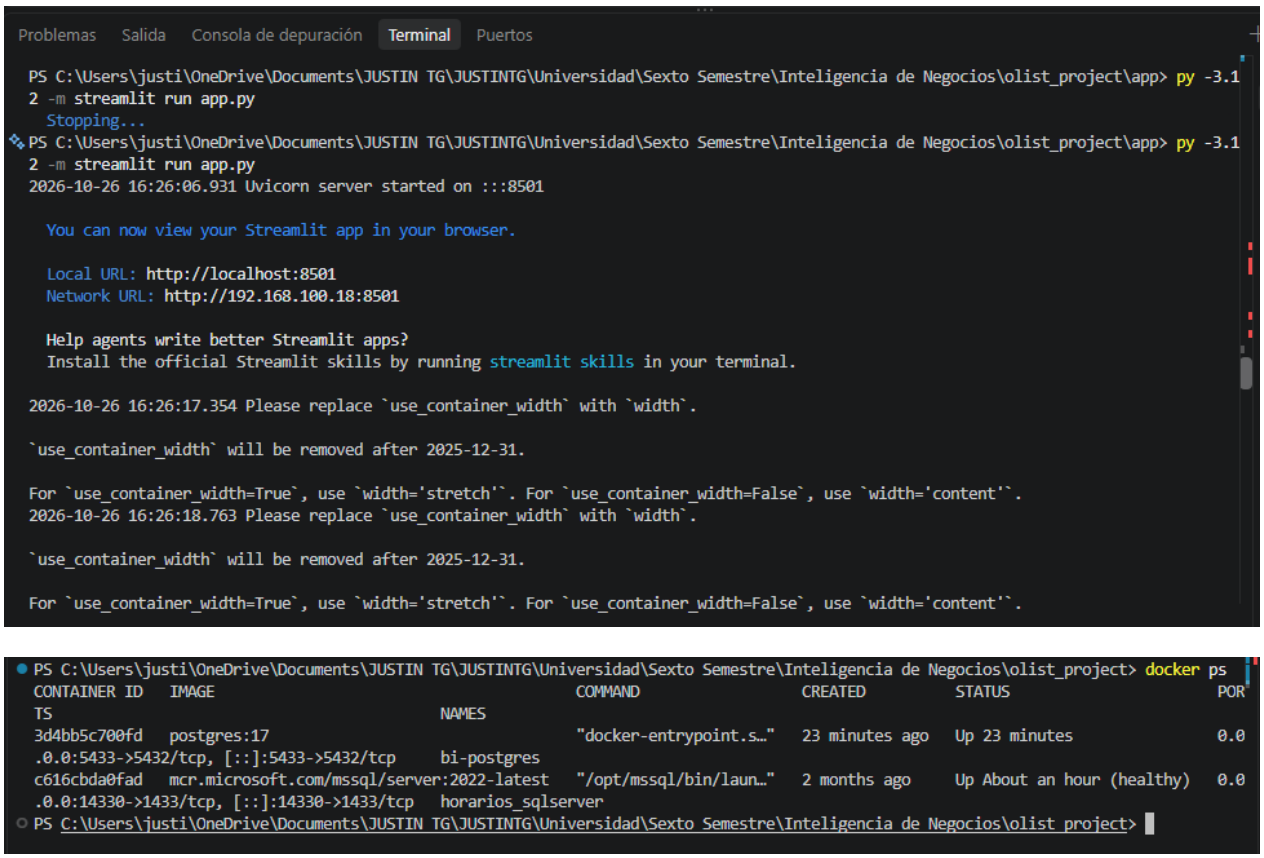


Figura 2. Ejecución del ETL y contenedor PostgreSQL activo.

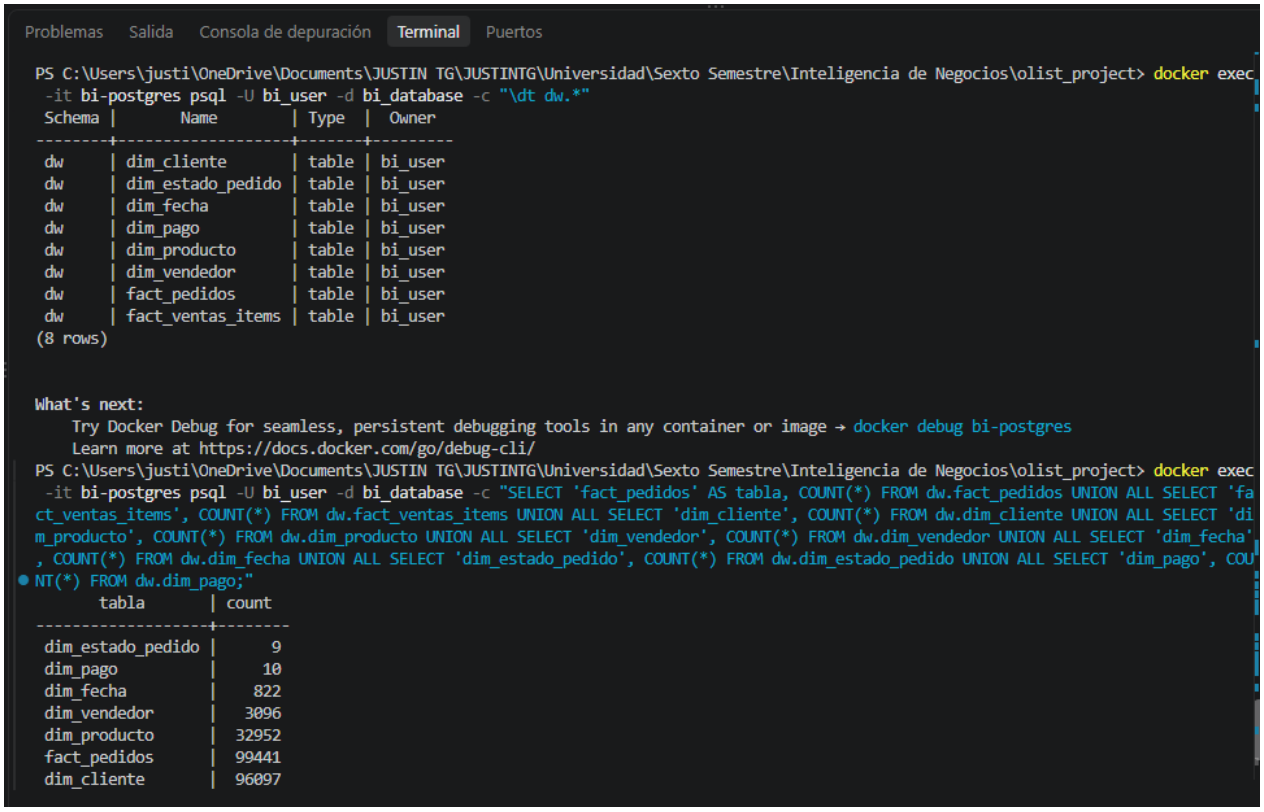


Figura 3. Tablas del Data Mart cargadas en PostgreSQL y sus conteos.

5.1 Resultados de validación (referencia)

Medida	Valor
Ingresos totales (suma price)	13.591.643,70
Ingresos sin cancelados (fn_kpi_ventas)	13.494.400,74
Pedidos entregados	96.478
Tasa entrega a tiempo	93,2 %
Tiempo promedio de entrega	12,5 días
Puntaje promedio reseñas	4,16
Vendedores activos	3.053
% ingresos top 10% vendedores	67,5 %

```
Problemas  Salida  Consola de depuración  Terminal  Puertos

sql/03_validaciones.sql -Encoding UTF8 | docker exec -i bi-postgres psql -U bi_user -d bi_databaseNegocios\olist_project>
SET
       tabla       | filas  | esperado | estado
-------------------+--------+----------+--------
 dim_fecha         |    822 |      822 | OK
 dim_cliente       |  96097 |    96097 | OK
 dim_producto      |  32952 |    32952 | OK
 dim_vendedor      |   3096 |     3096 | OK
 dim_estado_pedido |      9 |        9 | OK
 dim_pago          |     10 |       10 | OK
 fact_pedidos      |  99441 |    99441 | OK
 fact_ventas_items | 112650 |   112650 | OK
(8 rows)

             relacion             | huerfanos | estado
----------------------------------+-----------+--------
 fact_pedidos -> dim_estado       |         0 | OK
 fact_pedidos -> dim_pago         |         0 | OK
 fact_pedidos -> dim_fecha (compra) |       0 | OK
 fact_ventas_items -> dim_vendedor |        0 | OK
 fact_ventas_items -> dim_producto |        0 | OK
 fact_pedidos -> dim_cliente      |         0 | OK
 fact_ventas_items -> fact_pedidos |        0 | OK
(7 rows)

              indicador              | cantidad
-------------------------------------+----------
 Pedidos sin pago (pago_key=0)       |        1
 Productos sin categor?a (sin_categoria) |  610
 Pedidos delivered sin fecha de entrega |     8
 Pedidos sin ?tems                   |      775
 Pedidos sin fecha de entrega        |     2965
 Pedidos sin rese?a                  |      768
```

```
 ingresos_totales | flete_total | pedidos_con_items
------------------+-------------+-------------------
      13591643.70 |  2251909.54 |             98666
(1 row)

 pedidos_entregados | pct_a_tiempo | dias_promedio | puntaje_promedio
--------------------+--------------+---------------+------------------
              96478 |         93.2 |          12.5 |             4.09
(1 row)
```

Figura 4. Resultado de sql/03_validaciones.sql (integridad, nulos y totales).

```
PS C:\Users\justi\OneDrive\Documents\JUSTIN TG\JUSTINTG\Universidad\Sexto Semestre\Inteligencia de Negocios\olist_project> docker exec
 -it bi-postgres psql -U bi_user -d bi_database -c "SELECT * FROM dw.fn_kpi_ventas(); SELECT * FROM dw.fn_kpi_ventas(p_fecha_ini => '2
017-01-01', p_fecha_fin => '2017-12-31');"
  ingresos   |   flete    | pedidos | items  | ticket_promedio
-------------+------------+---------+--------+-----------------
 13494400.74 | 2241126.29 |   98199 | 112101 |          137.42
(1 row)

  ingresos   |   flete   | pedidos | items | ticket_promedio
-------------+-----------+---------+-------+-----------------
 6108492.27 | 982076.97 |   44375 | 50617 |          137.66
(1 row)
```

Figura 5. Función fn_kpi_ventas() ejecutada sin filtros y con filtro de fechas.

6. Mockup de la aplicación

El mockup representa la estructura prevista del dashboard: una barra lateral con los cuatro filtros (fecha inicio, fecha fin, categoría y estado del cliente) y tres vistas en pestañas, cada una con KPI, gráficos y tabla de detalle. Los valores marcados con # son ilustrativos en el mockup; los datos reales se muestran en las capturas de la aplicación.

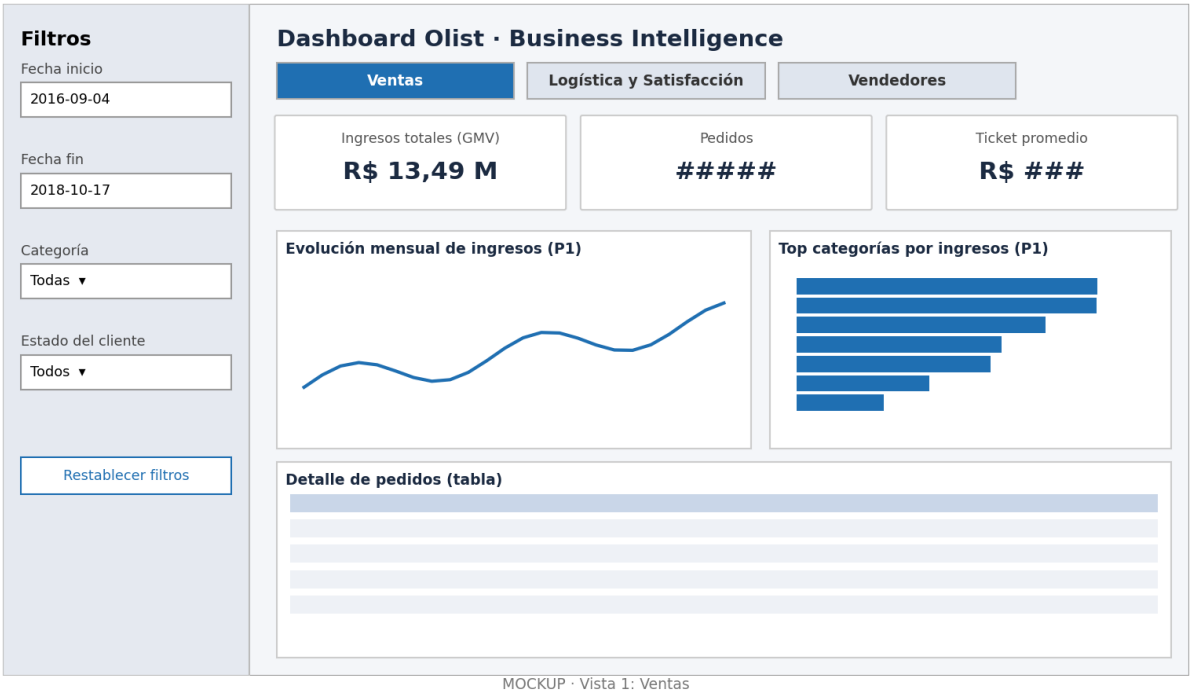


Figura 6. Mockup vista 1: Ventas (KPI de ingresos, evolución mensual, top categorías y detalle de pedidos).

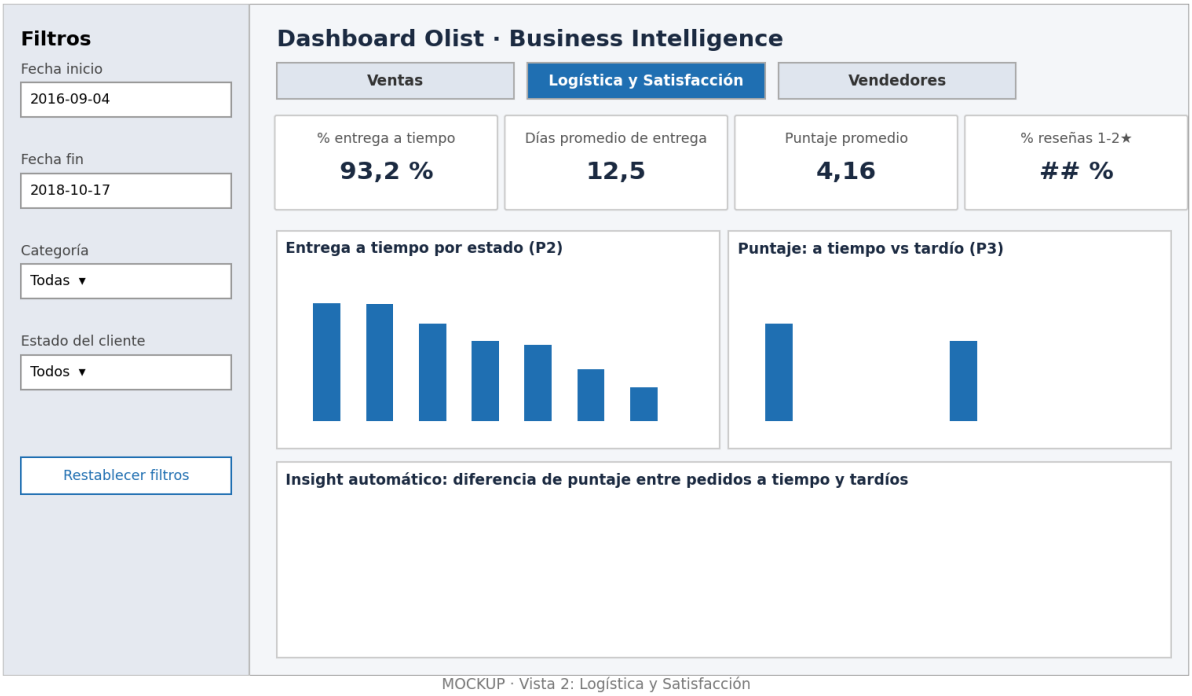


Figura 7. Mockup vista 2: Logística y Satisfacción (entrega a tiempo, puntaje y puntualidad).

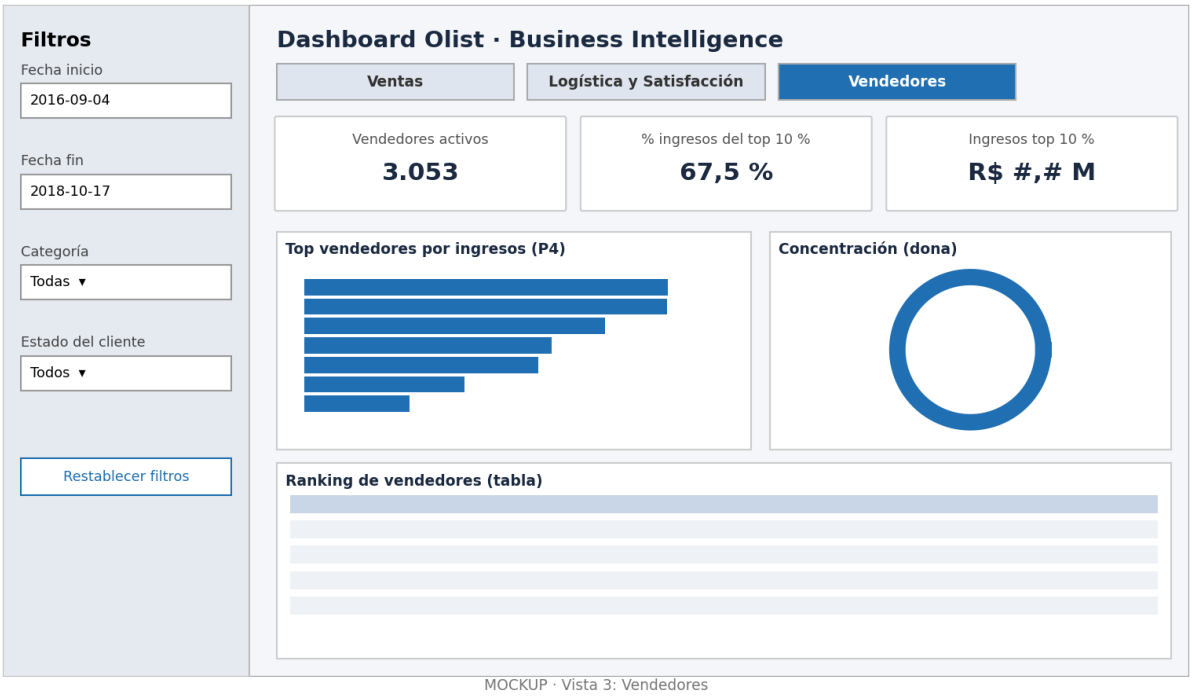


Figura 8. Mockup vista 3: Vendedores (concentración de ingresos y ranking).

7. Matriz de trazabilidad

Componente	Pregunta/KPI	Filtros	Objeto SQL
KPI Ingresos / Pedidos / Ticket	P1	Período, Categoría, Estado	fn_kpi_ventas()
Gráfico ingresos por mes	P1	Período, Categoría, Estado	fn_ventas_por_mes()
Gráfico top categorías	P1	Período, Estado	fn_top_categorias()
KPI % a tiempo / Días promedio	P2	Período, Estado	fn_kpi_logistica()
Gráfico entrega por estado	P2	Período	fn_entrega_por_estado()
KPI Puntaje / % negativas	P3	Período, Estado	fn_kpi_satisfaccion()
Gráfico puntaje vs puntualidad	P3	Período, Estado	fn_satisfaccion_vs_entrega()
KPI vendedores / concentración	P4	Período, Categoría	fn_concentracion_vendedores()
Gráfico top vendedores	P4	Período, Categoría	fn_top_vendedores()
Tabla detalle pedidos	P1-P3	Período, Estado	fn_detalle_pedidos()

8. Aplicación Python + Streamlit

La aplicación (app/app.py) se conecta a PostgreSQL mediante SQLAlchemy/psycopg2 (módulo db.py). Todos los KPI, gráficos y tablas se obtienen llamando a las funciones SQL del esquema dw. No se leen CSV para los indicadores finales.

Componentes implementados:

• 3 vistas (pestañas): Ventas, Logística y Satisfacción, Vendedores

• 3+ filtros interactivos: período (fecha inicio/fin), categoría de producto, estado del cliente

• KPI por vista (ingresos, pedidos, ticket, % a tiempo, días promedio, puntaje, concentración de vendedores)

• Gráficos con Plotly: línea de ingresos mensuales, barras de categorías, entrega por estado, puntaje vs puntualidad, top vendedores, dona de concentración

• Tabla de detalle de pedidos y ranking de vendedores

• Los filtros afectan realmente las consultas (parámetros a las funciones SQL)

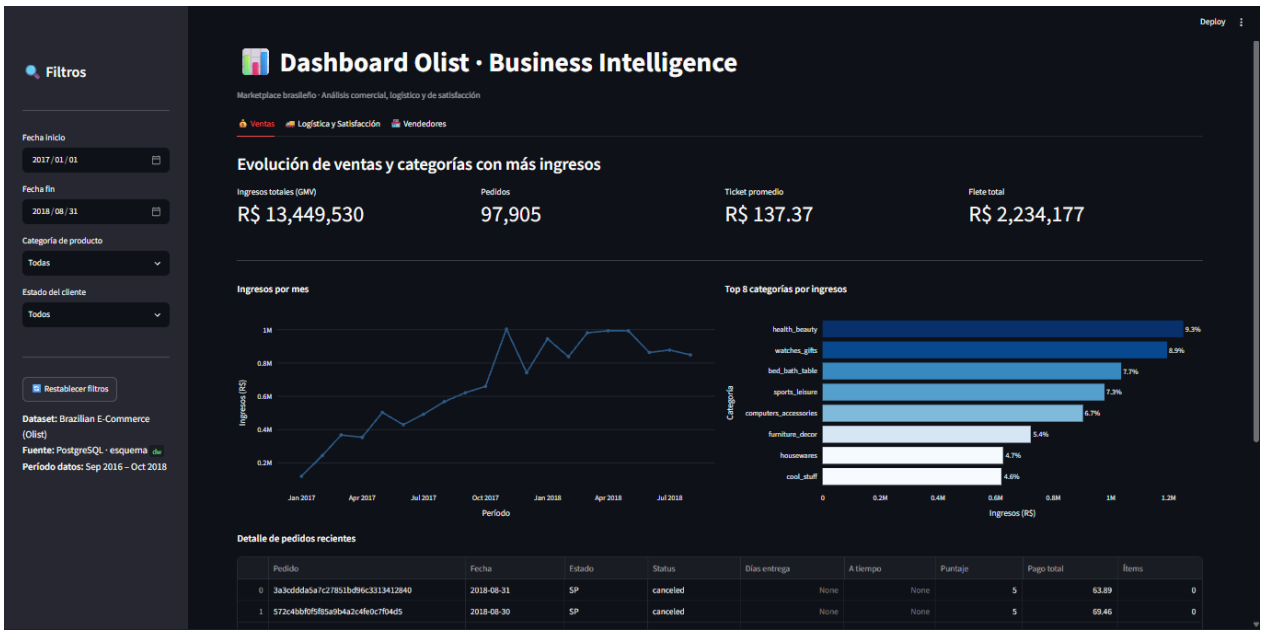


Figura 9. Aplicación Streamlit: vista Ventas.

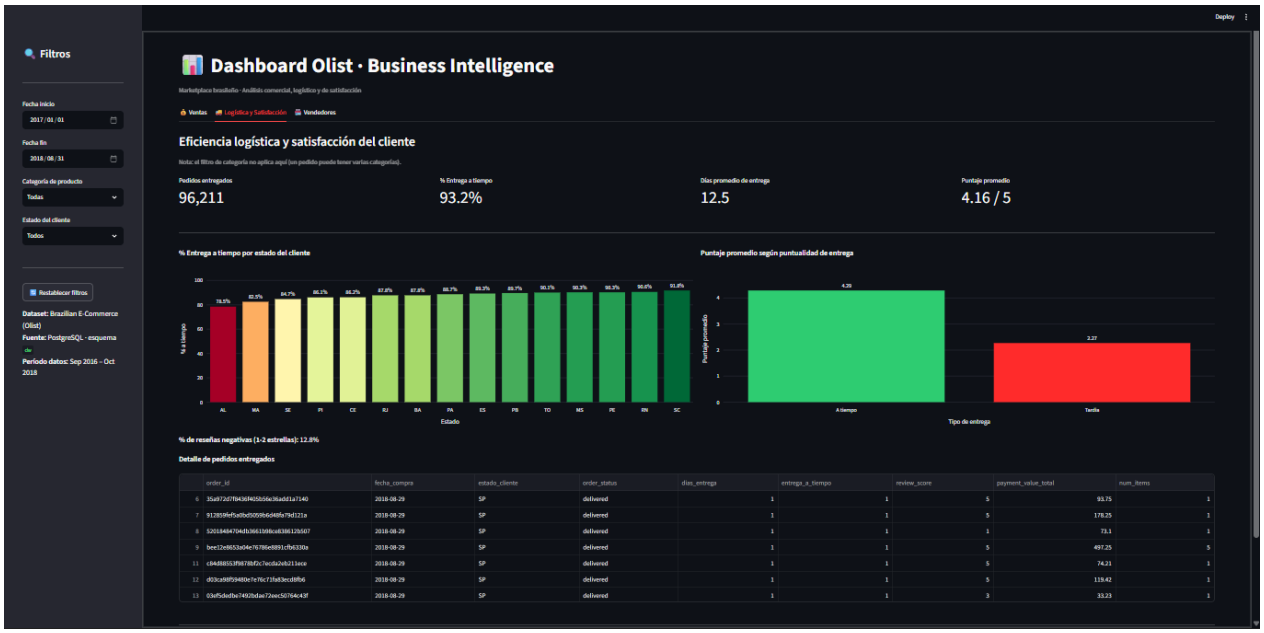


Figura 10. Aplicación Streamlit: vista Logística y Satisfacción.

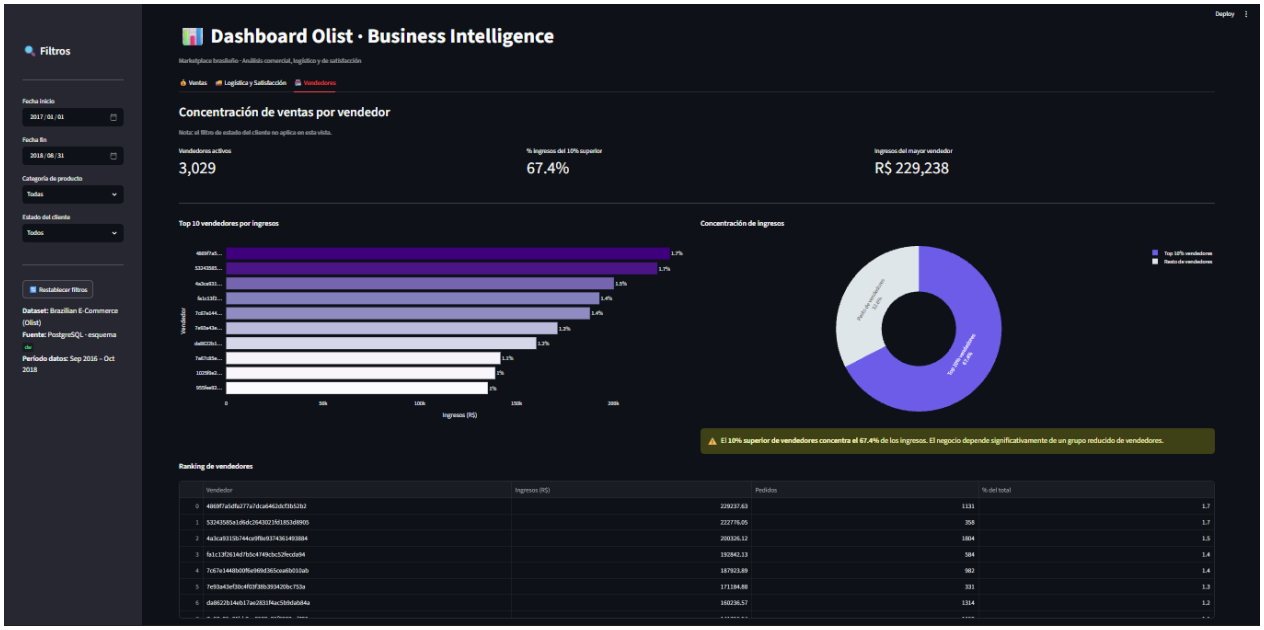
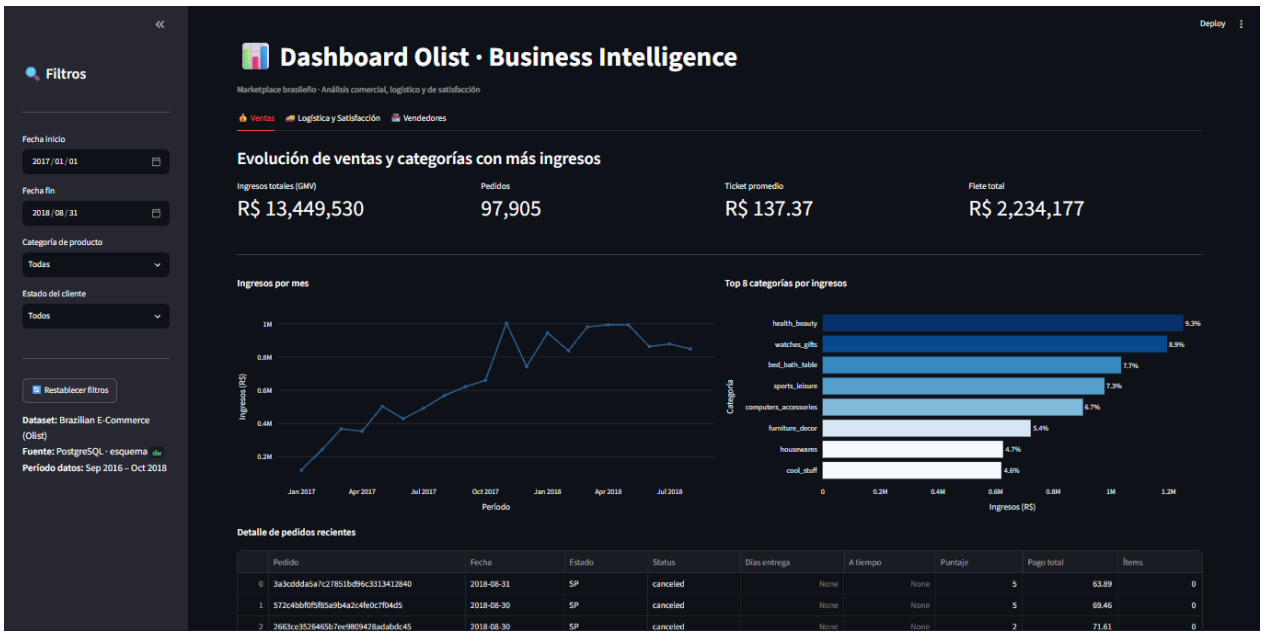


Figura 11. Aplicación Streamlit: vista Vendedores.



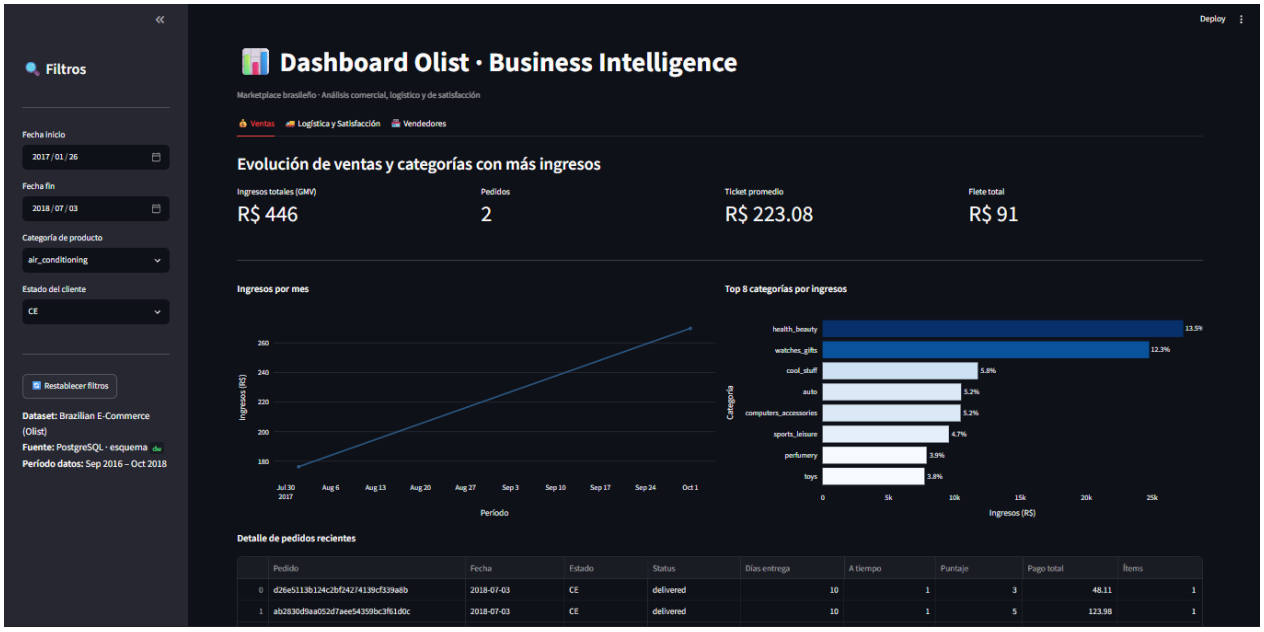


Figura 12. Efecto de los filtros sobre los KPI y gráficos (antes / después).

9. Evidencias de uso de Inteligencia Artificial

Uso 1: Diseño del modelo dimensional en constelación

Problema: Las medidas tienen distintos niveles de granularidad (ítem vs pedido). No estaba claro si usar una sola tabla de hechos o dos.

Prompt: "Tengo un dataset de e-commerce con order_items (precio por ítem) y orders (entrega y reseña por pedido). ¿Cómo diseño el modelo dimensional para no duplicar medidas?"

Solución de la IA: Propuso un esquema en constelación con fact_ventas_items y fact_pedidos compartiendo dimensiones.

Qué se utilizó: Se adoptó la constelación. Se adaptó agregando la fila 'sin dato' y las claves sustitutas.

Comprobación: Las validaciones de integridad dieron 0 huérfanos y los totales coincidieron con los calculados en Python.

Uso 2: Escritura de funciones SQL con filtros opcionales

Problema: Necesitaba funciones que aceptaran filtros de fecha, categoría y estado, devolviendo NULL = sin filtro.

Prompt: "Escribe una función PostgreSQL que calcule ingresos y ticket promedio con parámetros opcionales de fecha_ini, fecha_fin, categoria y estado_cliente."

Solución de la IA: Función LANGUAGE sql con COALESCE y condiciones (p_param IS NULL OR columna = p_param).

Qué se utilizó: La estructura de parámetros nombrados y el patrón de filtros opcionales en las 10 funciones.

Comprobación: SELECT * FROM fn_kpi_ventas() y con filtros devolvieron los mismos valores que el cálculo manual sobre los CSV.

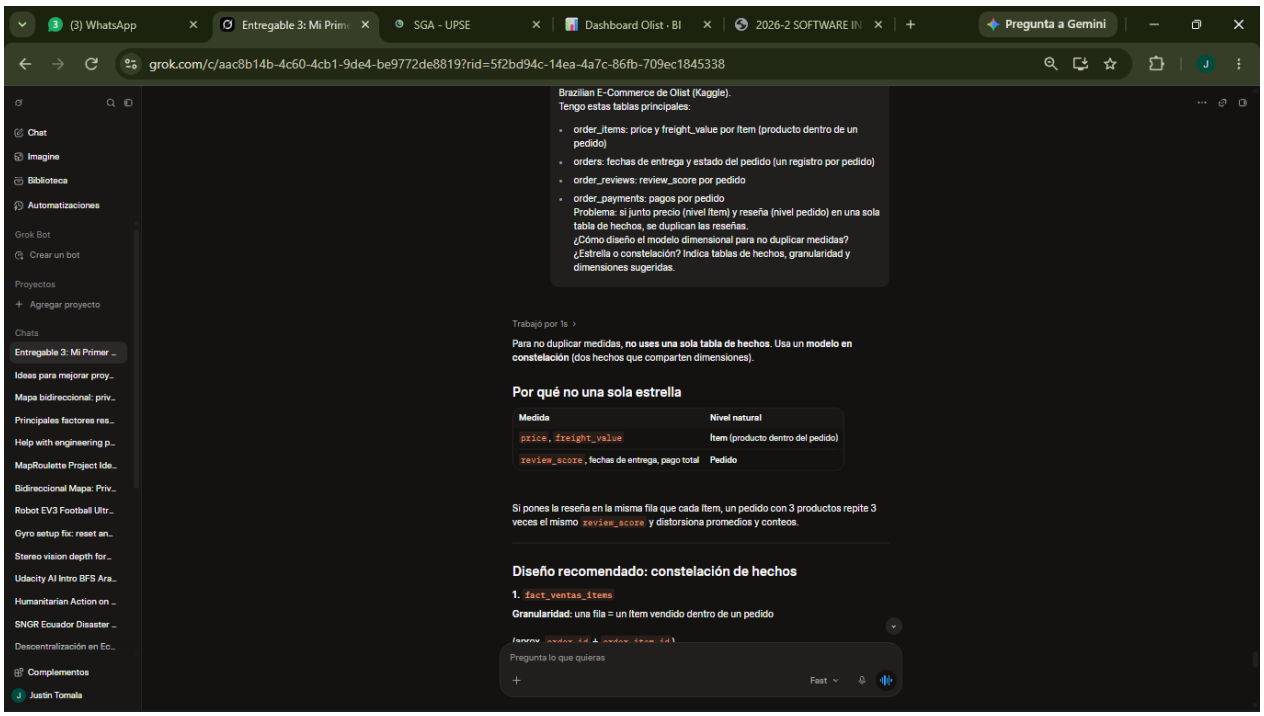
Uso 3: Conexión Streamlit–PostgreSQL y gráficos interactivos
Problema: Cómo conectar Streamlit a PostgreSQL de forma segura y construir gráficos que reaccionen a los filtros del sidebar.

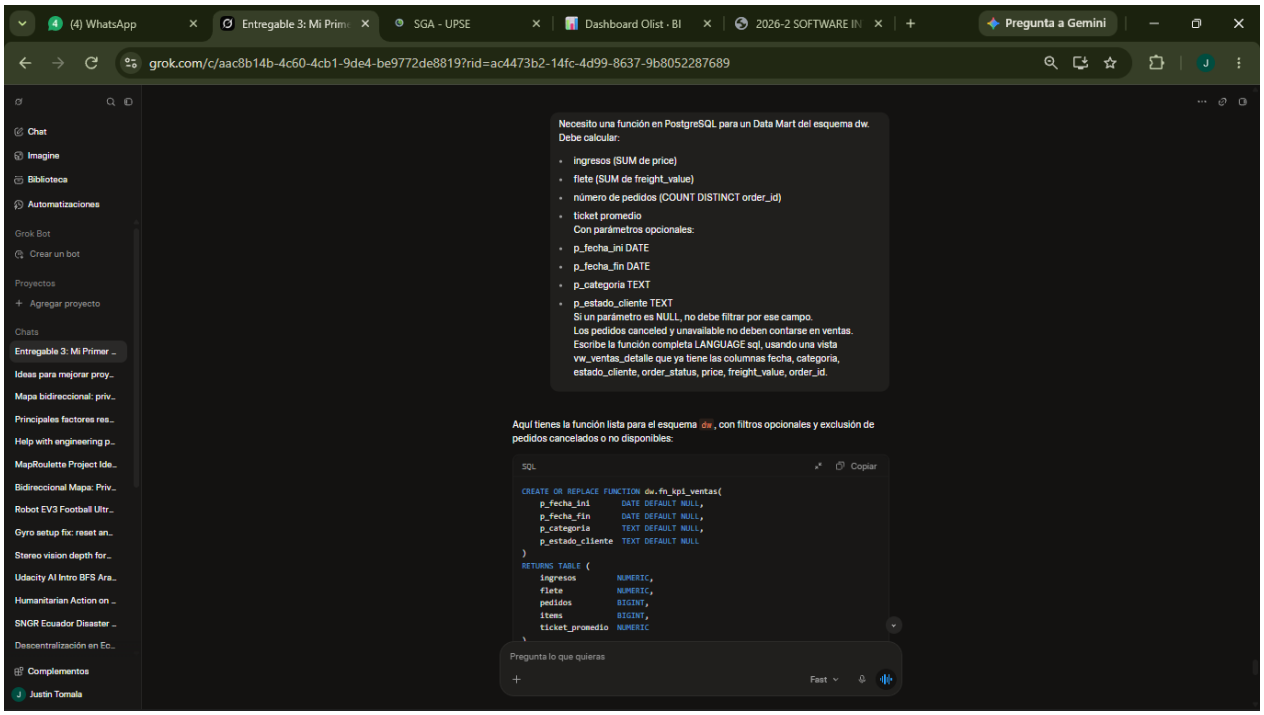
Prompt: "Ejemplo de app Streamlit que llama funciones SQL de PostgreSQL con parámetros de filtros de fecha y categoría, y muestra KPI + gráficos Plotly."

Solución de la IA: Módulo de conexión con SQLAlchemy, st.sidebar para filtros, call_fn() con parámetros nombrados, plotly.express para gráficos.

Qué se utilizó: La arquitectura db.py + app.py. Se adaptó el estilo visual y se añadieron insights automáticos (ej. diferencia de puntaje a tiempo vs tardío).

Comprobación: Al cambiar filtros en el dashboard, los KPI y gráficos se actualizan y coinciden con las consultas SQL directas.


(3) WhatsApp
Entregable 3: Mi Prim
SGA - UPSE
Dashboard Olist - BI
2026-2 SOFTWARE IN
Pregunta a Gemini
grok.com/c/aac8b14b-4c60-4cb1-9de4-be9772de8819?rid=5f2bd94c-14ea-4a7c-86fb-709ec1845338
Chat
Imagine
Biblioteca
Automatizaciones
Grok Bot
Crear un bot
Proyectos
Agregar proyecto
Chats
Entregable 3: Mi Primer ...
Ideas para mejorar proy...
Mapa bidireccional: priv...
Principales factores res...
Help with engineering p...
MapRoulette Project Ide...
Bidireccional Mapa: Priv...
Robot EV3 Football Ultr...
Gyro setup fix: reset an...
Stereo vision depth for...
Udacity AI Intro BFS Ara...
Humanitarian Action on ...
SNGR Ecuador Disaster ...
Descentralización en Ec...
Complementos
Justin Tomala
Brazilian E-Commerce de Olist (Kaggle).
Tengo estas tablas principales:
order_items: price y freight_value por ítem (producto dentro de un pedido)
orders: fechas de entrega y estado del pedido (un registro por pedido)
order_reviews: review_score por pedido
order_payments: pagos por pedido
Problema: si junto precio (nivel ítem) y reseña (nivel pedido) en una sola tabla de hechos, se duplican las reseñas.
¿Cómo diseño el modelo dimensional para no duplicar medidas?
¿Estrella o constelación? Indica tablas de hechos, granularidad y dimensiones sugeridas.
Trabajó por 1s
Para no duplicar medidas, no uses una sola tabla de hechos. Usa un modelo en constelación (dos hechos que comparten dimensiones).
Por qué no una sola estrella
Medida
Nivel natural
price, freight_value
Ítem (producto dentro del pedido)
review_score, fechas de entrega, pago total
Pedido
Si pones la reseña en la misma fila que cada ítem, un pedido con 3 productos repite 3 veces el mismo review_score y distorsiona promedios y conteos.
Diseño recomendado: constelación de hechos
1. fact_ventas_items
Granularidad: una fila = un ítem vendido dentro de un pedido
Pregunta lo que quieras
Fast


(4) WhatsApp
Entregable 3: Mi Prim
SGA - UPSE
Dashboard Olist - BI
2026-2 SOFTWARE IN
Pregunta a Gemini
grok.com/c/aac8b14b-4c60-4cb1-9de4-be9772de8819?rid=ac4473b2-14fc-4d99-8637-9b8052287689
Chat
Imagine
Biblioteca
Automatizaciones
Grok Bot
Crear un bot
Proyectos
Agregar proyecto
Chats
Entregable 3: Mi Primer ...
Ideas para mejorar proy...
Mapa bidireccional: priv...
Principales factores res...
Help with engineering p...
MapRoulette Project Ide...
Bidireccional Mapa: Priv...
Robot EV3 Football Ultr...
Gyro setup fix: reset an...
Stereo vision depth for...
Udacity AI Intro BFS Ara...
Humanitarian Action on ...
SNGR Ecuador Disaster ...
Descentralización en Ec...
Complementos
Justin Tomala
Necesito una función en PostgreSQL para un Data Mart del esquema dw.
Debe calcular:
ingresos (SUM de price)
flete (SUM de freight_value)
número de pedidos (COUNT DISTINCT order_id)
ticket promedio
Con parámetros opcionales:
p_fecha_ini DATE
p_fecha_fin DATE
p_categoria TEXT
p_estado_cliente TEXT
Si un parámetro es NULL, no debe filtrar por ese campo.
Los pedidos canceled y unavailable no deben contarse en ventas.
Escribe la función completa LANGUAGE sql, usando una vista vw_ventas_detalle que ya tiene las columnas fecha, categoria, estado_cliente, order_status, price, freight_value, order_id.
Aquí tienes la función lista para el esquema dw, con filtros opcionales y exclusión de pedidos cancelados o no disponibles:
SQL
Copiar
CREATE OR REPLACE FUNCTION dw.fn_kpi_ventas(
p_fecha_ini DATE DEFAULT NULL,
p_fecha_fin DATE DEFAULT NULL,
p_categoria TEXT DEFAULT NULL,
p_estado_cliente TEXT DEFAULT NULL
)
RETURNS TABLE (
ingresos NUMERIC,
flete NUMERIC,
pedidos BIGINT,
items BIGINT,
ticket_promedio NUMERIC
)
Pregunta lo que quieras
Fast

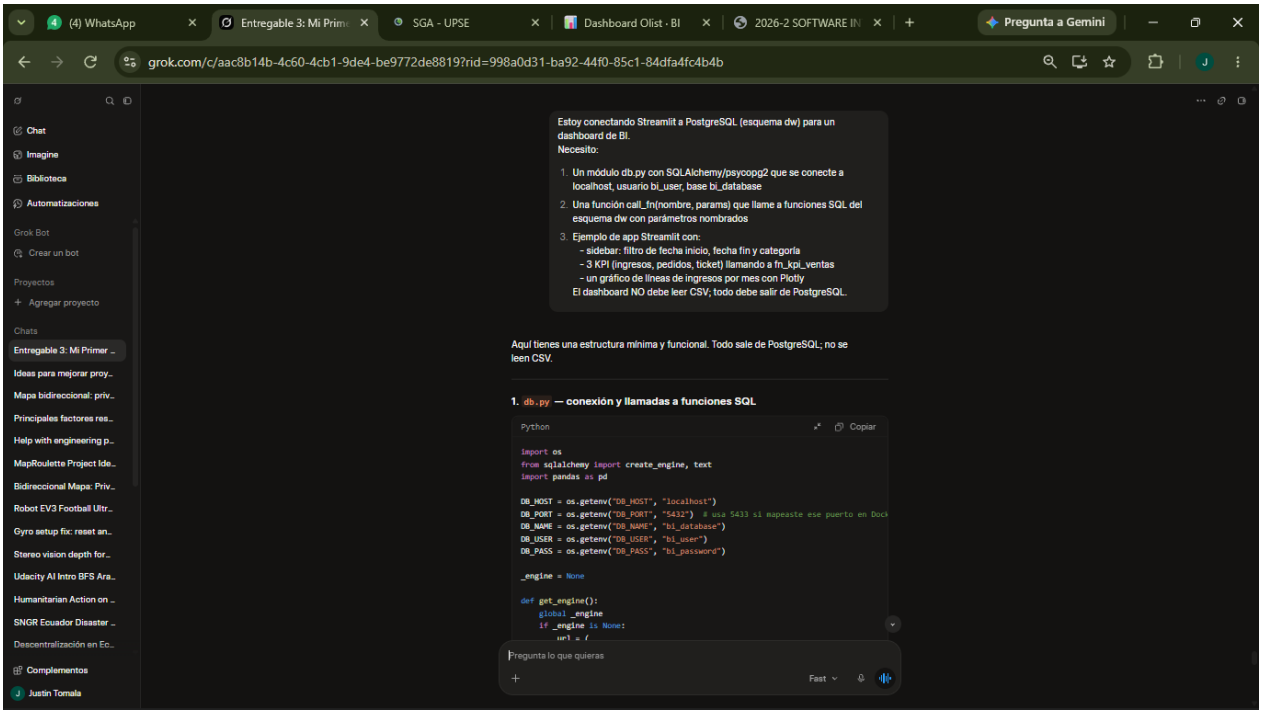


Figura 13. Evidencia de las conversaciones con la herramienta de IA.

10. Conclusión

Se construyó una solución de Business Intelligence de extremo a extremo: desde el dataset real de Olist, pasando por la limpieza reproducible en Python, el Data Mart dimensional en PostgreSQL (constelación de hechos), las vistas y funciones SQL, hasta el dashboard interactivo en Streamlit alimentado exclusivamente desde la base de datos.

Los resultados permiten apoyar decisiones de crecimiento de ventas (categorías y regiones), mejora de la operación de entregas (estados con más retrasos), atención al cliente (relación retraso–satisfacción) y gestión de vendedores (concentración del 67,5 % de ingresos en el 10 % superior).

Arquitectura final: DATOS → TRANSFORMACIÓN → DATA MART → SQL → PYTHON → STREAMLIT → ANÁLISIS → DECISIÓN.

Link del repositorio: https://github.com/JustinTG11/Mi-Primer-Dashboard

Link del video: Video de Mi primer Dashboard o https://upse-my.sharepoint.com/my?id=%2Fpersonal%2Fjustin%5Ftomalagonzalez0433%5Fupse%5Fedu%5Fec%2FDocuments%2FVideo%20de%20Mi%20primer%20Dashboard&viewid=abcbb24a%2Da4bc%2D4102%2D9b63%2D9192a8993b62&ga=1